Introducción a Ciencia de Datos

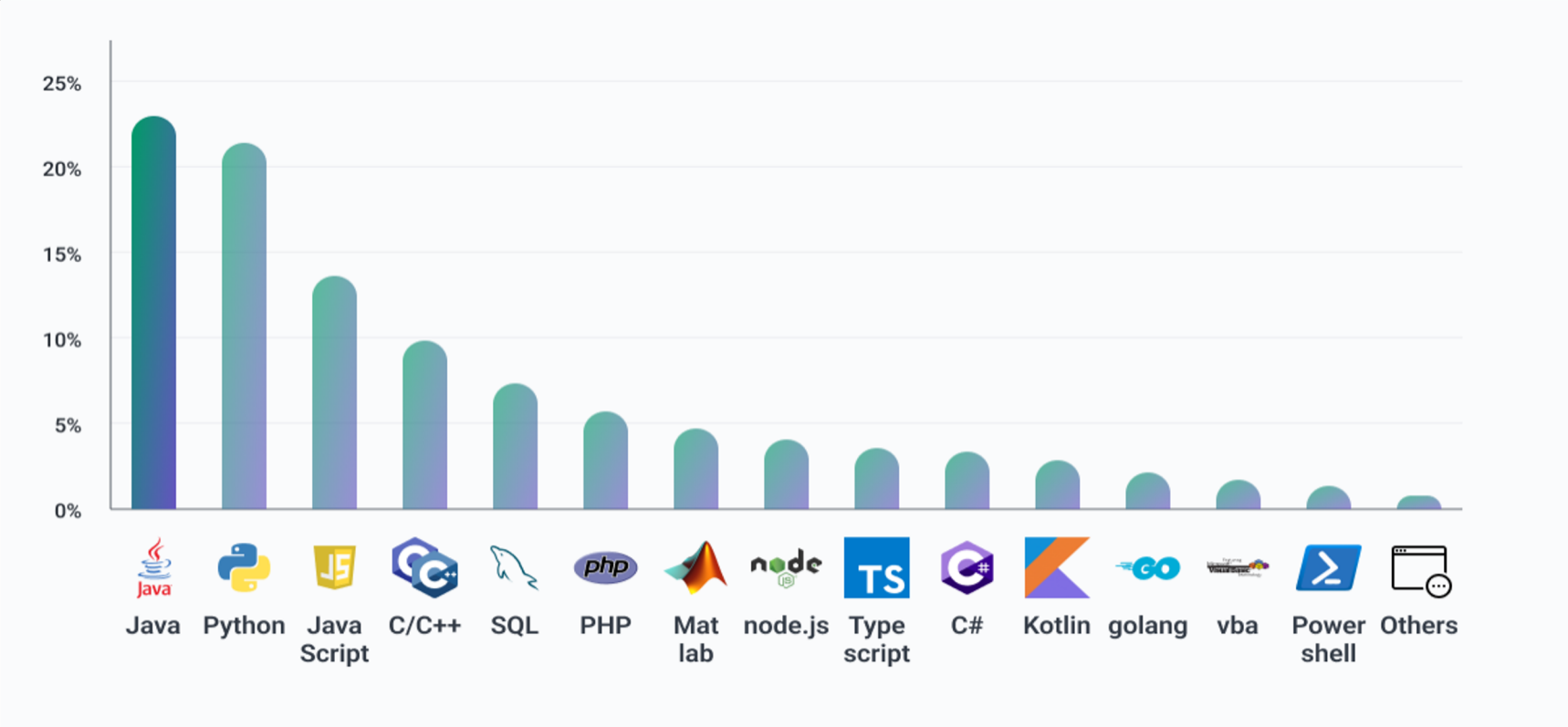
Introducción a Python

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

¿Por Qué Python?

- Legibilidad y simplicidad
- Open Source
  - ✓ Gran ecosistema de bibliotecas y frameworks
  - ✓ Comunidad activa y soporte
- Cross-platform
- Fuerte desarrollo en una variedad de áreas
  - ✓ Desarrollo de aplicaciones con interfaz gráfica
  - ✓ Análisis y Ciencia de Datos
  - ✓ Simulación numérica
  - ✓ Desarrollo Web e Internet
  - ✓ Automatización de Tareas

Trending Technologies: Most Googled Programming Languages Across the World in 2023



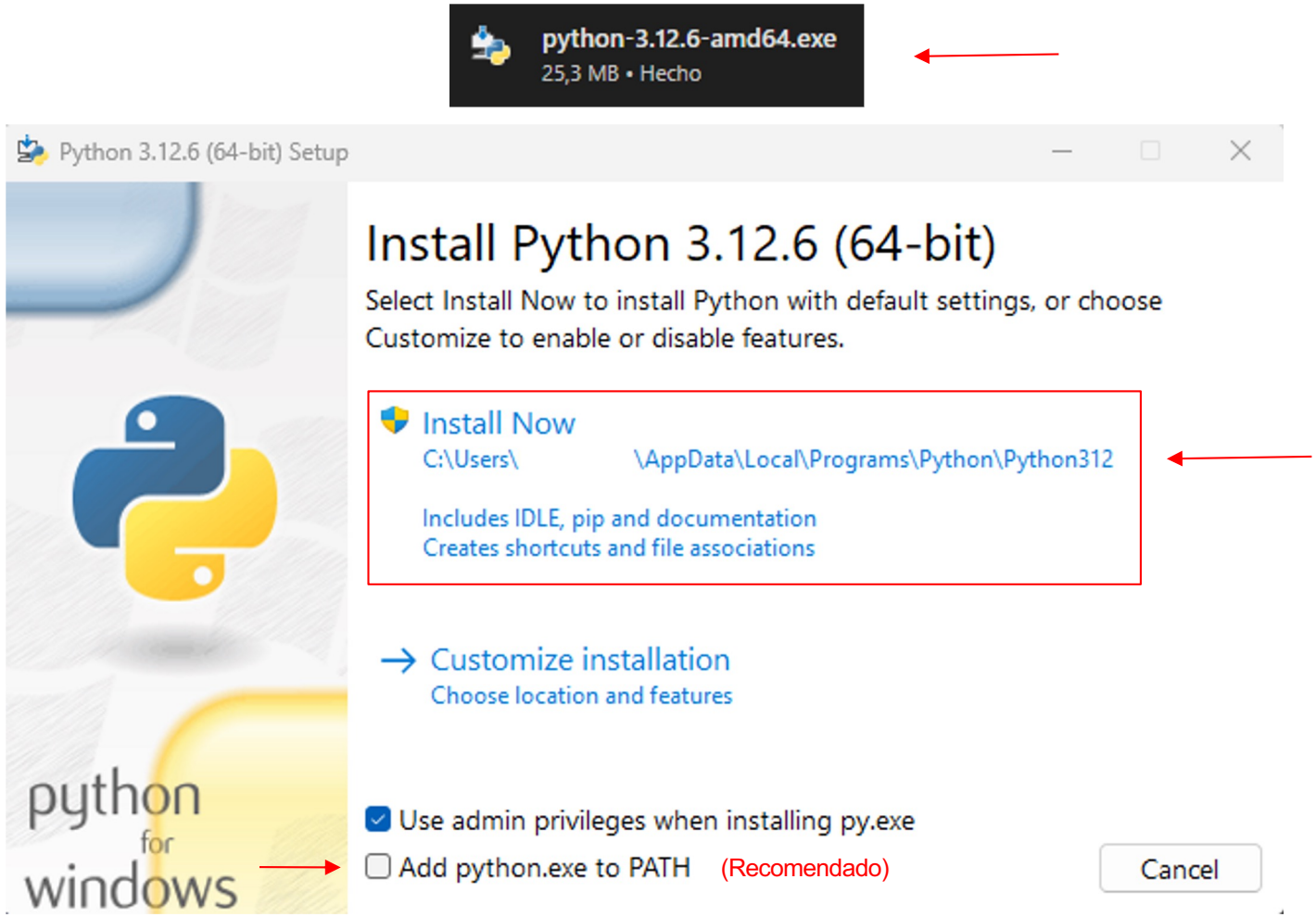
fuente: https://softjourn.com/insights/is-java-still-used

Instalación de Python

- Tradicional: https://www.python.org/

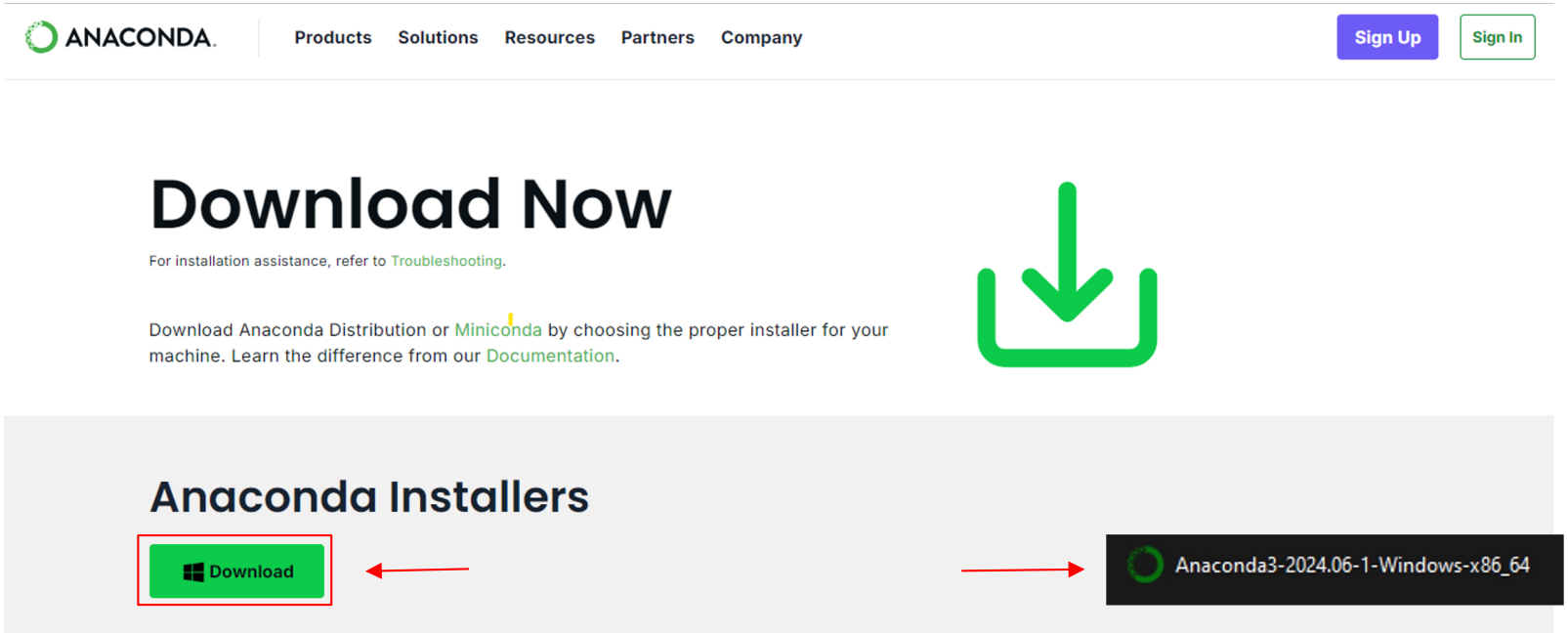


Esta es la instalación básica y directa de Python desde su sitio oficial. Al instalarlo de esta manera, obtenés el lenguaje, el intérprete, y algunas herramientas estándar, como el gestor de paquetes pip para instalar bibliotecas.

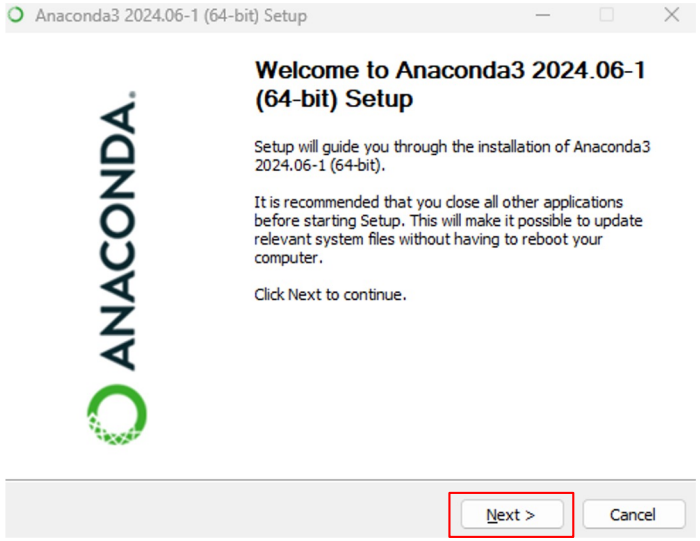


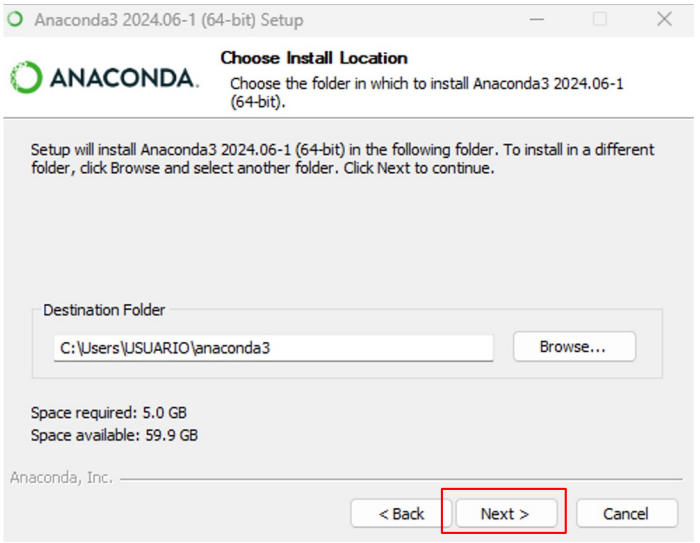
✓ Una vez instalado, se puede ejecutar en la consola de windows ***python --version*** para chequear si termino correctamente

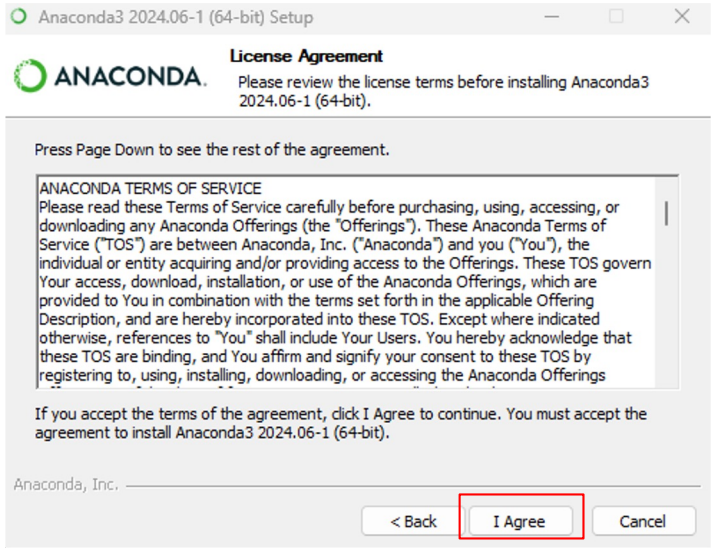
- Anaconda: https://www.anaconda.com/download/success

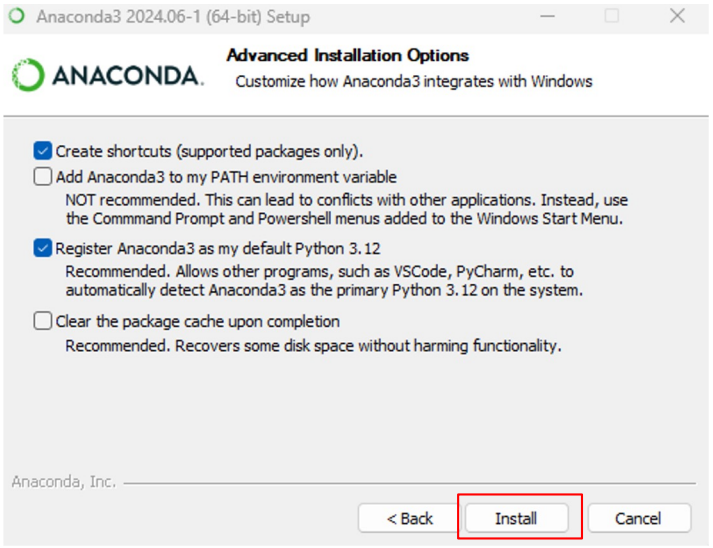


Al instalar Anaconda, se obtiene Python, el gestor de entornos y paquetes ***conda***, y varias bibliotecas preinstaladas.

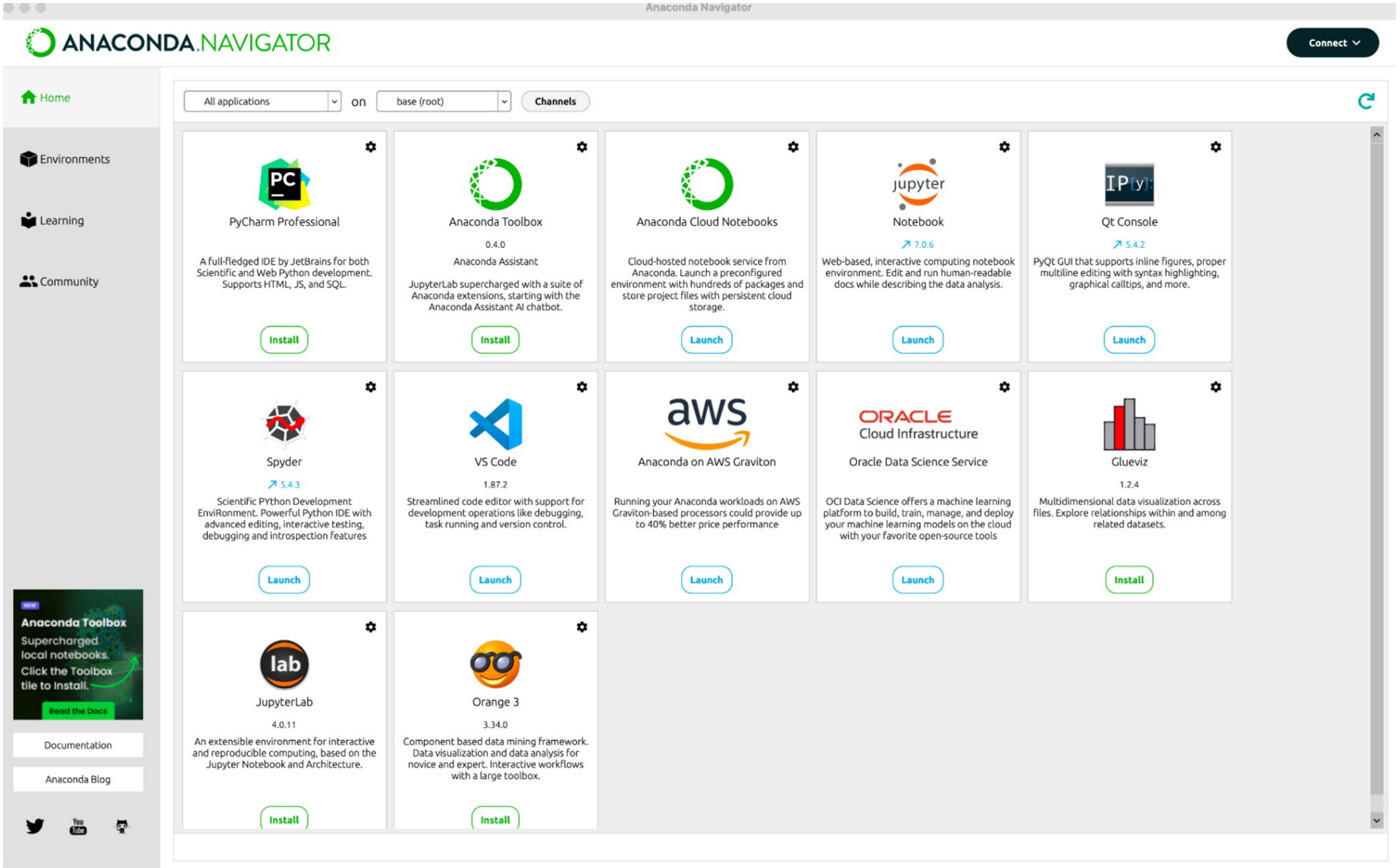








UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES



UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

- Ventajas:
  - ✓ **Gestión de dependencias**: Muchos paquetes científicos dependen de versiones específicas de otros paquetes. Navigator facilita asegurar que cada paquete tenga las dependencias correctas.
  - ✓ **Manejo de múltiples versiones**: Los científicos de datos a menudo utilizan varias versiones de un mismo paquete, y Navigator permite gestionar diferentes entornos para separar esas versiones.
  - ✓ **Interfaz gráfica amigable**: Navigator ofrece una interfaz gráfica que permite trabajar con paquetes y entornos sin necesidad de escribir comandos en la terminal.
  - ✓ **Instalación y actualización sencilla**: Permite buscar, instalar y actualizar paquetes dentro de entornos de manera visual y sencilla.
- Desventajas:
  - X **Manejo de recursos**: Al incluir tantas herramientas, puede consumir más recursos del sistema en comparación con un entorno más liviano y personalizado.
  - X **Dependencia de conda**: Aunque *conda* es poderoso, no siempre es compatible con todos los paquetes de *pip*, lo que puede causar conflictos o complicaciones al usar paquetes externos o más recientes.
  - X **Menos control**: Al usar Anaconda, puedes tener menos control sobre qué bibliotecas están instaladas inicialmente, lo que puede hacer que sea más difícil mantener un entorno mínimo y eficiente si solo necesitas un subconjunto de las herramientas incluidas.

Nociones Fundamentales de Python

¿Cómo ejecutar un programa?

- **Scripts (.py):**

  ✓ Un **script** en Python es un archivo de texto plano que contiene líneas de código que se ejecutan de principio a fin de manera secuencial.

  ✓ Los scripts están diseñados para ser ejecutados completamente, sin necesidad de interacción o ejecución en partes.

  ✓ Editores más comunes **Spyder**, **Atom** y **Visual Studio Code**

- **Notebooks (.ipynb):**

  ✓ Son entornos interactivos que permiten combinar código, texto, gráficos y visualizaciones en un solo documento.

  ✓ Las herramientas para editar notebooks más comunes son **Jupyter Notebook**, **Google Colab** y **Visual Studio Code**

Tipos de objetos, operaciones básicas y formatos

Empecemos usando a Python como una calculadora y de paso veamos la importancia de prevenir la ambigüedad en un programa:

```
[in]  3 + 3 * 2
```

```
[out] 9
```

```
[in]  (3 + 3) * 2
```

```
[out] 12
```

Python tiene que considerar un par de cosas.

- ¿Que son **3** y **2**?
- ¿Qué operación indica el símbolo **+** sobre **3** y **3**?
- ¿Puedo realizar la operación en este caso?

Tipos de objetos

Veamos otra operación realizada en Python:

```
[in]  'Hasta el infinito ' + 'y más allá!'
[out] Hasta el infinito y más allá!
```

Realizó otra operación con el +, *concatenó* dos textos.

Python sabe que a distintos tipos de datos les corresponden distintas operaciones.

Algunos tipos de datos son:

- Los **números enteros**, que están etiquetados **int** (de *integer*).
- Los **números "reales"** (limitados a una cantidad de decimales), que se guardan como **float**.
- Los datos de **texto**, siempre entre comillas (simples ' ' o dobles " "), son *strings*: **str**.
- los **booleanos**, **bool**, que tienen dos valores posibles: Verdadero (**True**) o Falso (**False**).

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

En Python, podemos obtener el tipo de un objeto de la siguiente forma:

```
[in]  type(0.1) # Consulto el tipo del objeto
[out] float
```

Cualquier texto escrito luego de **#** Python lo toma como un **comentario**

- **type** es una función que Python tiene predefinida, muy útil para saber el tipo de dato con el cual estamos trabajando.

Las **funciones** realizan alguna tarea específica sobre los argumentos que se les dan entre paréntesis.

- Otra función predefinida usamos mucho es **print**, que imprime todos sus argumentos:

```
[in]  print('El significado de todo es', 42)
[out] El significado de todo es 42
```

Operaciones sobre Números

Veamos algunas operaciones numéricas que Python tiene definidas:

- ✓ **+** suma dos números
- ✓ **-** los resta
- ✓ Se usa ***** para multiplicar
- ✓ Se divide con **/**
- ✓ **//** indica una división entera (por ejemplo, **10 // 8** daría **1**)
- ✓ Para potencias, ****** es el símbolo de "elevado a".
- ✓ Y el operador **%** (módulo) me da el resto de una división (por ejemplo, **10 % 8** da **2**).

Veamos un ejemplo:

```
[in]  8/2 # Hago una división
[out] 4.0
```

En vez de devolver un **int**, la división devuelve un **float**.

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Guardar datos: ¿Que son las variables?

- Una variable es un nombre que asignamos a un valor que Python guarda en su memoria para poder reutilizarlo más tarde en nuestro código.
- Nos permiten ejecutar programas con más de una línea de código
- Las variables pueden contener diferentes tipos de datos, como números, cadenas de texto, listas, funciones, entre otros.
- El nombre debe comenzar con una letra o un guión bajo, y no pueden incluir espacios ni comenzar con números. Además, Python distingue entre mayúsculas y minúsculas, por lo que `variable` y `Variable` son distintas.

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

- Para asignar un dato a una variable, usamos el signo **=**.

```
[in]    x = 5
        print(x)
        y = 2**x
        print(y)
        x = "Un texto"
        print(x)
        print(y)
```

```
[out]   5
        32
        Un texto
        32
```

Observamos que:

- ✓ Python ejecuta el código línea por línea de arriba hacia abajo, evaluando y asignando los valores en ese orden.
- ✓ Las variables funcionan como "cajas" que almacenan datos. Cuando asignamos un valor a una variable (por ejemplo, `x = 5`), estamos guardando ese valor en la "caja" llamada `x`
- ✓ Se pueden sobrescribir las variables. Al reasignar `x`, su valor anterior se pierde, pero `y` no cambia porque fue calculada antes.

Cómo dar formato al texto

Para imprimir un texto junto a el valor de una variable, podemos utilizar una línea de este estilo:

```
[in]  print('La siguiente variable', nombre_variable, 'es muy importante.')
```

Una manera de reescribir esta línea de forma más elegante es usando un ***f-string*** (format string).

- ✓ El método se basa en poner una **f** al principio de cualquier *string*, y luego colocar la variable que queramos pasar al formato dentro de corchetes "{}". Veamos a los f-string's en acción.

```
[in]  print(f"2 + 3 es igual a {2+3}")
[out] 2 + 3 es igual a 5
```

Linters y estilos de codificación

Un **linter** en Python es una herramienta para identificar:

- Errores sintácticos y lógicos.
- Violaciones del estilo de codificación (como las reglas definidas en **PEP 8**).
- Código ineficiente o propenso a errores.
- Variables sin usar o mal definidas.

Linters populares en Python:

✓ **Pylint**: No solo detecta errores, sino que también evalúa la calidad general del código y ofrece recomendaciones para mejorarlo.

✓ **Flake8**: Es más ligero que Pylint y se enfoca principalmente en errores de sintaxis y estilo.

✓ **Pyflakes**: Es una herramienta simple que se centra en encontrar errores de lógica como variables no definidas o sin usar.

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

¿Qué es PEP 8?

PEP 8 es una guía de estilo con un conjunto de reglas que organizan tu código para que sea más claro y fácil de entender.

Algunas reglas básicas de PEP 8:

- ✓ Usa 4 espacios por cada nivel de **Indentación**.
- ✓ No hagas **líneas** de código **muy largas**.
- ✓ Deja un **espacio alrededor** de los **operadores** .
- ✓ Para las **variables**, usa **nombres claros** y en minúsculas, con palabras separadas por guiones bajos.
- ✓ **Comenta** tu código para que quede claro qué hace y porqué lo hace, pero sin abusar.

Mas Informacion: https://peps.python.org/pep-0008/

Ejemplos:

- Indentación

```
[in] # Mala práctica
     def greet():
      print("Hola, mundo!")  # Aquí hay solo 1 espacio de indentación

     # Buena práctica
     def greet():
         print("Hola, mundo!")  # Aquí se usan 4 espacios de indentación
```

- Espacios alrededor de operadores

```
[in] # Mala práctica
     a=5+2

     # Buena práctica
     a = 5 + 2  # Agregamos espacios alrededor del signo igual y del más
```

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

- Longitud de líneas

```
[in] # Mala práctica
     mi_diccionario = {"nombre": "Juan", "apellido": "Pérez", "edad": 30, "ciudad": "Buenos Aires", "pr

     # Buena práctica
     mi_diccionario = {
         "nombre": "Juan",
         "apellido": "Pérez",
         "edad": 30,
         "ciudad": "Buenos Aires",
         "profesión": "Ingeniero",
         "empresa": "Tech Corp",
         "intereses": ["programación", "deportes", "música"]
     }
```

- Nombres de variables

```
[in] # Mala práctica
     e = 23
     a = 1.82

     # Buena práctica
     edad = 23
     altura = 1.82  # Ahora es más fácil entender qué representan las variables
```

- Comentarios

```
[in] # Mala práctica
     print("Hola") # Esto imprime "Hola"

     # Buena práctica
     print("Hola") # Imprimimos un saludo para el usuario
```

- Ejemplo general

```
[in] # Mala práctica
     print(15+(15*(21/100))-(15*0.3))


     # Buena práctica
     precio = 15
     porcentaje_impuesto = 21
     porcentaje_descuento = 30

     # Calculo cuanto de impuesto tengo que sumar
     impuesto = precio * (porcentaje_impuesto / 100)

     # Calculo cuanto de descuento tengo que restar
     descuento = precio * (porcentaje_descuento / 100)

     precio_total = precio + impuesto - descuento

     print(precio_total)
```

Booleanos y condicionales

Booleanos

- Una variable del tipo booleana sólo tiene dos valores posibles: True y False.
- Se comportan siguiendo la lógica proposicional

Algunos ejemplos:

```
[in]  print(3 > 4)# ¿3 es mayor que 4?
[out] False
```

```
[in]  print(3**2 != 2**3) # ¿3² es distinto de 2³?
[out] True
```

```
[in]  # ¿Está la letra 'o' en el string 'Euler'?
      print('o' in 'Euler')
[out] False
```

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Operadores De Comparación

- a **>** b verifica si a es **mayor** que b
- a **<** b verifica si a es **menor** que b
- a **>=** b verifica si a es **mayor o igual** que b
- a **<=** b verifica si a es **menor o igual** que b
- a **!=** b verifica si a es **distinto** que b
- a **==** b verifica si a es **igual** que b

Extra: a **in** b verifica si a está contenido en la secuencia b

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Condicionales: *if* - *elif* - *else*

Veamos cómo escribir un código que sólo ejecute en caso de que se cumpla alguna **condición**:

```
[in]   # Ejemplo
       llueve = False

       if llueve:
           print("¡No te olvides el paraguas!")
```

Observaciones:

- `llueve` es la **condición** en este caso, la línea con el print solo se ejecutará en caso de que `llueve` sea True.
- Las líneas que inician con if terminan con **:**
- Las instrucciones para el caso verdadero están corridas 4 espacios respecto al if

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

¿Que devuelve el siguiente programa?:

```
[in]  llueve = True

      if llueve:
          print("¡No te olvides el paraguas!")

      print('No te olvides la SUBE para el colectivo')
```

```
[out]  ¡No te olvides el paraguas!
       No te olvides la SUBE para el colectivo.
```

Otro ejemplo:

```
[in]  num = 5
      if num > 0:
          print(f'{num} es positivo.')

      if num <  10:
          print(f'{num} es menor que 10.')
```

```
[out] 5 es positivo.
      5 es menor que 10.
```

Sigamos ahora condiciones que no son independientes:

```
[in]    num = -42
        if num > 0:
            print(f'{num} es positivo.')
        else:
            print(f'{num} es cero o negativo.')
```

```
[out]   -42 es cero o negativo.
```

El else se ejecuta en caso de que no se cumpla la condición del if que lo antecede.

Extra: podemos usar **or** y **and** para combinar condiciones, también podemos negarlas con **not**.

```
[in]    if (num > 0) and not(num > 42):
            print(f'{num} es positivo y no es mayor a 42.')
```

Ejemplo:

```
[in] saldo_sube = 10  # pesos
     pasaje_colectivo = 400  # pesos

     # Si alcanza para la ida y la vuelta...
     if saldo_sube > 2*pasaje_colectivo :
         print('Alcanza para dos viajes.')

     # Si no alcanza...
     else:
         diff = 2*pasaje_colectivo  - saldo_sube
         print(f'Para ir y volver, deberías cargarle al menos {diff}.')
```

```
[out] Para ir y volver, deberías cargarle al menos 790.
```

El programa es bueno, pero puede mejorar, sería fundamental poder discernir entre tres escenarios excluyentes:

- ✓ El saldo alcanza para la ida y la vuelta. Cargamos otro día.
- ✓ El saldo alcanza para la ida, pero no para la vuelta. Cargamos en un kiosco cercano.
- ✓ El saldo no alcanza para la ida ni para la vuelta. Cargamos por casa.

Para estos casos, debemos utilizar el elif (abreviatura de else if):

```
[in] saldo_sube = 450  # pesos
     pasaje_colectivo = 400  # pesos
     diff = 2*pasaje_colectivo - saldo_sube

     # Si alcanza para la ida y la vuelta...
     if saldo_sube > 2 * pasaje_colectivo:
         print('Alcanza para dos viajes.')

     # Si no, si alcanza para un solo viaje...
     elif saldo_sube > pasaje_colectivo:
         print('Alcanza para un solo viaje.')
         print(f'En un kiosco cercano, carga al menos {diff}.')

     # Si no alcanza para nada...
     else:
         print('No alcanza ni para un viaje.')
         print(f'Cargale por casa al menos {diff}.')
```

```
[out] Alcanza para un solo viaje.
      En un kiosco cercano, carga al menos 350.
```

La estructura completa queda:

```
[in] ... <resto_del_codigo>

     if <condicion_1>:
         <instrucciones para caso verdadero>

     elif <condicion_2>:
         <instrucciones si el caso anterior es falso>

     elif <condicion_3>:
         <instrucciones si el caso anterior es falso>

     ...(elif condicion_4 etc)

     else:
         <instrucciones si todos los casos anteriores fueron falsos>

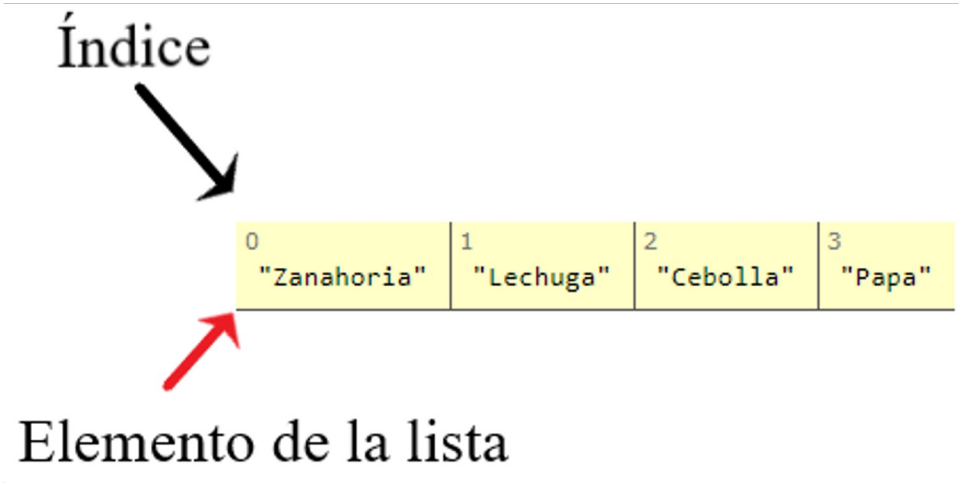
     ... <resto_del_codigo>
```

Listas

Listas

- Sirven para agrupar elementos de forma eficiente
- Las **listas** se definen abriendo y cerrando **corchetes**
- Los elementos se separan con **comas**.

```
[in]  verduras = ['Zanahoria', 'Lechuga', 'Cebolla', 'Papa']
```



- Con los **corchetes** nos podemos referir a cualquier elemento de la **lista**.

```
[in]  verduras[0]  # Pedimos el primer elemento de la lista
[out] 'Zanahoria'
```

- En Python se empieza a numerar desde el 0, también podemos empezar a contar de atrás para adelante.

```
[in]   print(verduras[3])  # Pedimos el cuarto elemento de la lista
       print(verduras[-1])  # Pedimos el último elemento de la lista
```

```
[out]  Papa
       Papa
```

- Las **listas** pueden tener elementos de distintos tipos adentro.

```
[in]   comida = 'Milanesa'
       lista = [1.42, 22, 'Alfombra', True, comida, 'Jorge', "1234", 12.1, False]
```

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Operaciones y métodos aplicables a listas

- ✓ **+ Concatena** dos listas en una nueva (**-** no tiene operaciones asignadas)
- ✓ ***** sirve para **repetir** los elementos de una lista ([1 , 2, 3] ***** 3 devuelve [1, 2, 3, 1, 2, 3, 1, 2, 3])
- ✓ Se usa **:** dentro de los corchetes de una lista para **partirla**..

```
[in]    lista = [1, 2, 3, 4, 5]
        print(f'Primera porcion: {lista[:3]}')
        print(f'Porcion del medio: {lista[1:3]}')
```

```
[out]   Primera porcion: [1, 2, 3]
        Porcion del medio: [2, 3]
```

- ✓ **in** verifica si un elemento está en la lista. (`2 in [ 1, 2, 3 ] devuelve True`)
- ✓ **len**(lista) devuelve la cantidad de elementos en una lista.

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Los métodos se pueden utilizar de la siguiente manera: `objeto.metodo(argumentos)`

- ✓ **append**(elemento) Añade un elemento al final de la lista.

```
[in]   lista = [1, 2, 3]
       lista.append(4)
       print(lista)
       lista.append([4, 5, 6])
       print(lista)
```

```
[out]  [1, 2, 3, 4]
       [1, 2, 3, 4, [4, 5, 6]]
```

- ✓ **remove**(elemento) Elimina la primera aparición de un elemento.
- ✓ **index**(elemento) Devuelve el índice del primer elemento con un valor específico.
- ✓ **count**(elemento) Cuenta cuántas veces aparece un elemento en la lista
- ✓ **sort**() Ordena la lista (Se puede especificar el orden)

Ciclos for y while

Ciclos: For

Hagamos un código para mostrar todos los elementos de una lista

```
[in]   print(verduras[0])
       print(verduras[1])
       print(verduras[2])
       print(verduras[3])
```

- Un ciclo **for** permite correr código muchas veces sin que tengamos que hacerlo a mano.

```
[in]  for n in range(0,4):  # Arranca desde 0 y no incluye al 4
          print(verduras[n])
```

- Lo que se escribe luego del **for** , los **:** y los **4 espacios de indentación** se ejecuta, cambia el valor de n y se vuelve a ejecutar.

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Simplificamos el código anterior

```
[in]  for n in range(len(verduras)):
          print(verduras[n])
```

Otra forma de iterar por los elementos de una lista:

```
[in]  for elemento in verduras:
          print(elemento)
```

- ✓ La variable `n` o `elemento` cumplen el rol de *iterador*
- ✓ Se le puede poner cualquier nombre
- ✓ Sigue creada después del ciclo

```
[in]  lista = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
      suma = 0
      for i in lista:
          suma = suma + i
          #suma += i # Esta es otra manera de sumar más compacta
      print(suma)
```

```
[out]  55
```

Veamos casos donde el iterador son dos variables:

Usando **enumerate()**:

```
[in]  for indice, valor in enumerate(verduras):
          print(f"Índice: {indice}, Valor: {valor}")
```

```
[out]  Índice: 0, Valor: Zanahoria
       Índice: 1, Valor: Lechuga
       Índice: 2, Valor: Cebolla
       Índice: 3, Valor: Papa
```

Usando **zip()**:

```
[in]  nombres = ['Ana', 'Luis', 'Pedro']
      edades = [23, 34, 19]

      for nombre, edad in zip(nombres, edades):
          print(f"{nombre} tiene {edad} años")
```

```
[out]  Ana tiene 23 años
       Luis tiene 34 años
       Pedro tiene 19 años
```

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Ciclos: While

- El comando **while** permite otra forma de iterar.
- El **while** no crea un *iterador*, sino que tenemos que definirlo en el código.

```
[in]  verduras = ['Zanahoria', 'Lechuga', 'Cebolla', 'Papa'
      indice = 0 # Definimos el iterador

      while indice < len(verduras):
          print(verduras[indice])

          indice += 1
```

- El propósito del **for** es **iterar un objeto**, con tamaños fijos, como listas o diccionarios.
- El propósito del **while** es correr el código hasta que se **cumpla una condición**

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Otras Estructuras de datos en Python

Tuplas

- Las **tuplas** son similares a las listas, pero **inmutables**
- Puedes crear una **tupla** usando **paréntesis**

```
[in] tupla = (1, 2, 3)
     tupla_sin_parentesis = 1, 2, 3 # No recomendado
```

- Podés acceder a los elementos de una tupla utilizando índices y soportan algunos métodos de listas

```
[in] print(tupla[1])
     print(tupla.index(2))
```

```
[out] 2
      1
```

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

- **Desempaquetado** de tuplas:

```
[in]   a, b, c = tupla
       print(a, b, c)
```

```
[out] 1 2 3
```

- Usos más comunes:
  - ✓ Almacenamiento de datos constantes
  - ✓ Claves de diccionarios
  - ✓ Retorno de múltiples valores en funciones

Diccionarios

- Los **diccionarios** son colecciones de pares clave-valor
- Puedes crear un **diccionario** usando **llaves** y separando claves y valores con **dos puntos** o con **dict()**:

```
[in]  diccionario = {
      "nombre": "Juan",
      "edad": 30,
      "ciudad": "Buenos Aires"
      }
```

```
[in]  diccionario = dict(nombre="Juan",
                         edad=30,
                         ciudad="Buenos Aires")
```

- Las claves deben ser **inmutables** y **únicas**.

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

- Acceder a los elementos de un **diccionario**:
  - ✓ Utilizar su clave entre corchetes **[ ]**
  - ✓ Usar el método **get()**

```
[in]  print(diccionario["nombre"])
      print(diccionario.get("profesión", "No disponible"))
```

```
[out] Juan
      No disponible
```

- Modificación de valores

```
[in]  diccionario["edad"] = 31  # Modifica el valor existente
      diccionario["profesión"] = "Ingeniero"  # Agrega un nuevo par clave-valor

      print(diccionario)
```

```
[out] {'nombre': 'Juan', 'edad': 31, 'ciudad': 'Buenos Aires', 'profesión': 'Ingeniero'}
```

- **Métodos** útiles de los diccionarios

  ✓ **keys()**: Devuelve una vista de todas las claves del diccionario.

```
[in]  print(diccionario.keys())
[out] dict_keys(['nombre', 'edad', 'ciudad', 'profesión'])
```

  ✓ **values()**: Vista de todos los valores.

```
[in]  print(diccionario.values())
[out] dict_values(['Juan', 32, 'Córdoba', 'Ingeniero'])
```

  ✓ **items()**: Vista los pares clave-valor.

```
[in]  print(diccionario.items())
[out] dict_items([('nombre', 'Juan'), ('edad', 32), ('ciudad', 'Córdoba'),('profesión', 'Ingeniero')])
```

- ✓ **pop(**elemento**)**: Elimina una clave y devuelve su valor.
- ✓ **clear():** Elimina todos los elementos del diccionario
- ✓ **update(**diccionario**)**: Actualiza el diccionario con otro diccionario o pares clave-valor.

```
[in]  diccionario.update({"ciudad": "Córdoba", "edad": 32})
      print(diccionario)
```

```
[out] {'nombre': 'Juan', 'edad': 32, 'ciudad': 'Córdoba', 'profesión':
'Ingeniero'}
```

- Iterar sobre los ítems de un diccionario

```
[in]  for clave, valor in diccionario.items():
          print(f"{clave}: {valor}")
```

```
[out] nombre: Juan
      edad: 32
      ciudad: Córdoba
      profesión: Ingeniero
```

Otra forma:

```
[in]  for clave in diccionario.keys():
          print(f"{clave}: {diccionario[clave]}")
```

- Usos más comunes:
  - ✓ Almacenamiento de datos estructurados de **indexación** rápida
  - ✓ Filas de tablas (Última clase)

Sets

- Un **set** es una colección desordenada de elementos **únicos** y **mutables**.
- Podés crear un set utilizando **llaves** o la función **set()**:

```
[in] conjunto = {1, 2, 3, 4}
     conjunto_sin_duplicados = set([1, 2, 2, 3, 3, 3, 4])

     print(conjunto)
     print(conjunto_sin_duplicados)
```

```
[out] {1, 2, 3, 4}
      {1, 2, 3, 4}
```

- Los elementos en un set **no tienen un orden fijo**, no podés acceder a ellos por índice, como en una lista.

- **Métodos** útiles de los sets
  - ✓ **add(**elemento**)**: Agrega un elemento al set.
  - ✓ **remove(**elemento**)**: Elimina un elemento, tira error si el elemento no está.
  - ✓ **discard(**elemento**)**: Elimina un elemento, no tira error si no está.
  - ✓ **union(**set**)**: Devuelve la unión entre sets.
  - ✓ **intersection(**set**)**: Devuelve la intersección entre sets.
  - ✓ **difference(**set**)**: Devuelve los elementos que están en el primer set pero no en el segundo.
  - ✓ **clear()**: Elimina todos los elementos del set.
- Usos más comunes:
  - ✓ Eliminación de duplicados
  - ✓ Operaciones matemáticas de conjuntos

Definición de Funciones

Funciones

- Una **función** es un bloque de código **reutilizable** que realiza una tarea específica.
- Para definirla se utiliza **def**, el **nombre** de la función, los **parámetros** (si tiene) entre **paréntesis** y luego **dos puntos**.

```
[in] def saludar():
         print("Hola, bienvenido a la clase de Python!")

     saludar()
```

```
[out] Hola, bienvenido a la clase de Python!
```

Parámetros y Variables

Definamos una función con parámetros:

```
[in] def saludar_persona(nombre):
        print(f"Hola, {nombre}!")

saludar_persona("Nicolas") # Otra forma: Saludar_persona(nombre = "Nicolas")
```

```
[out] Hola, Nicolas!
```

Podemos definir parámetros con un valor predeterminado.

```
[in]  def saludar_persona(nombre = "Nicolas"):
          print(f"Hola, {nombre}!")

      saludar_persona()
      saludar_persona("Facundo")
```

```
[out] Hola, Nicolas!
      Hola, Facundo!
```

Funciones con múltiples parámetros:

```
[in] def sumar(a, b, c):
         return a + b +
c
```

Variables globales y locales:

```
[in] def duplicar(x):
    return(2*x)

x = 1

print(f'x local: {duplicar(x = 1)}')
print((f'x global: {x}'))
```

```
[out] x local: 2
      x global: 1
```

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Variables globales dentro de funciones:

```
[in] def duplicar_z():
         global x
         x *= 2
```

- Los objetos **mutables** (como listas, diccionarios o sets) **pueden modificarse** dentro de una función sin necesidad de declarar la variable como global.
- Los objetos **inmutables** (como enteros, strings o tuplas) **no se pueden modificar** directamente.

```
[in] def agregar_cuatro(lista):
         lista.append(4)

     mi_lista = [1, 2, 3]
     agregar_cuatro(mi_lista)
     print(mi_lista)
```

```
[out] [1, 2, 3, 4]
```

```python
[in] def calcular_precio_total(productos: list[tuple[str, float]], impuesto: float = 0.21) -> float:
    """
    Calcula el precio total de una lista de productos aplicando un impuesto.
    Parámetros:
    productos (list[tuple[str, float]]): Lista de tuplas donde cada tupla contiene
        el nombre del producto (str) y su precio (float).
    impuesto (float): El porcentaje de impuesto a aplicar, por defecto 21%.
    Retorna:
    float: El precio total de todos los productos, incluyendo el impuesto.
    """
    total_sin_impuesto = sum( [precio for _, precio in productos] )
    total_con_impuesto = total_sin_impuesto * (1 + impuesto)
    return total_con_impuesto
```

```
def calcular_precio_total(productos: list[tuple[str, float]], impuesto: float=0.21) -> float
```

Abrir en la pestaña Ver código fuente

Calcula el precio total de una lista de productos aplicando un impuesto.
Parámetros:
productos (list[tuple[str, float]]): Lista de tuplas donde cada tupla contiene
el nombre del producto (str) y su precio (float).
impuesto (float): El porcentaje de impuesto a aplicar, por defecto 21%.
Retorna:
float: El precio total de todos los productos, incluyendo el impuesto.

```
calcular_precio_total()
```

Librerías

Librerías, Importación e Instalación

- Las **librerías** son colecciones de **código preescrito** que ofrecen **funciones** y **módulos reutilizables** para resolver problemas específicos.
- Para usar una librería, primero debes **instalarla** y luego **importarla**.
- Las librerías se instalan con el gestor de paquetes **pip** (en la cmd)

  ✓ `pip install nombre_libreria` instala.

  ✓ `pip install nombre_libreria==version` instala una version específica.

  ✓ `pip install --upgrade nombre_libreria` actualiza.

  ✓ `pip uninstall nombre_libreria` desinstala .

  ✓ `pip list` chequeo las librerías en el entorno

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Importación:

- Las librerías se importan con la palabra clave **import**
- Hay distintas formas de hacerlo:

```
[in] import math
     print(math.sqrt(16))
```

```
[in] from math import sqrt
     print(sqrt(16))
```

```
[in] import math as m
     print(m.sqrt(16))
```

```
[in] from math import *
     print(sqrt(16))
```

Ejemplos de librerías **preinstaladas**

- ✓ **math**
- ✓ **datetime**
- ✓ **re**
- ✓ **random**
- ✓ **os**
- ✓ **sys**

Ejemplos librerías que necesitan **Instalación**

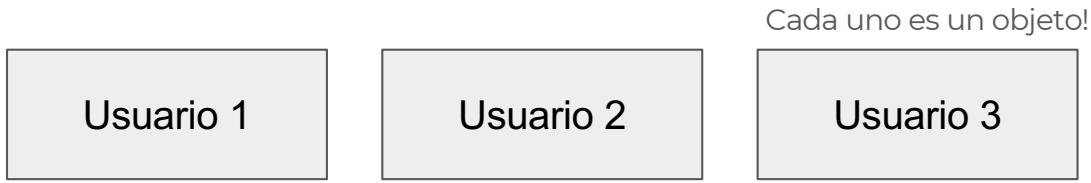
- ✓ **numpy**
- ✓ **pandas**
- ✓ **matplotlib**
- ✓ **scikit-learn**

Clases y objetos

Programación orientada a objetos

La Programación Orientada a Objetos (POO) es un paradigma de programación que organiza el código en **objetos**.

Los objetos son como "entidades" que tienen **propiedades** (atributos) y **comportamientos** (métodos o funciones).



Beneficios:

- Facilita la reutilización del código.
- Ayuda a organizar y estructurar mejor los programas.
- Mejora la legibilidad y mantenimiento del código.

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Conceptos Básicos de POO

Clase

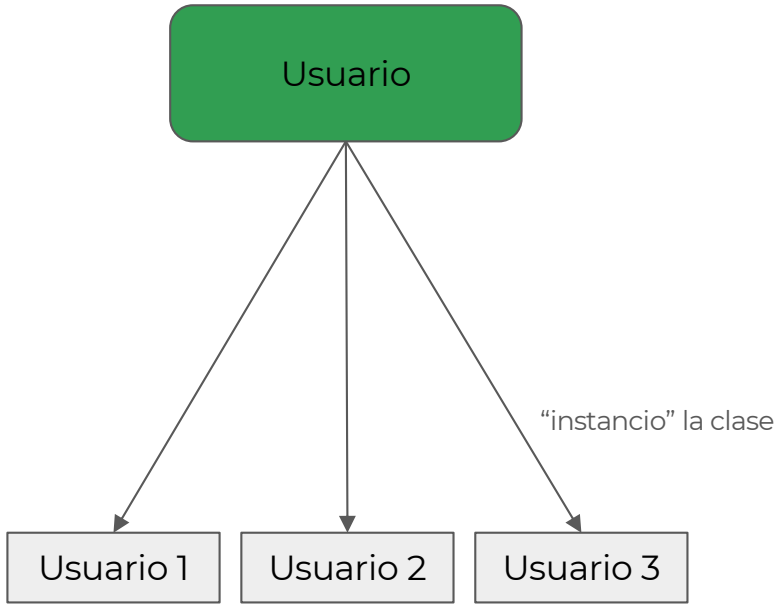
Es el molde o plantilla para crear objetos. Define los atributos y métodos que tendrán los objetos.

Usuario

Conceptos Básicos de POO

Objeto

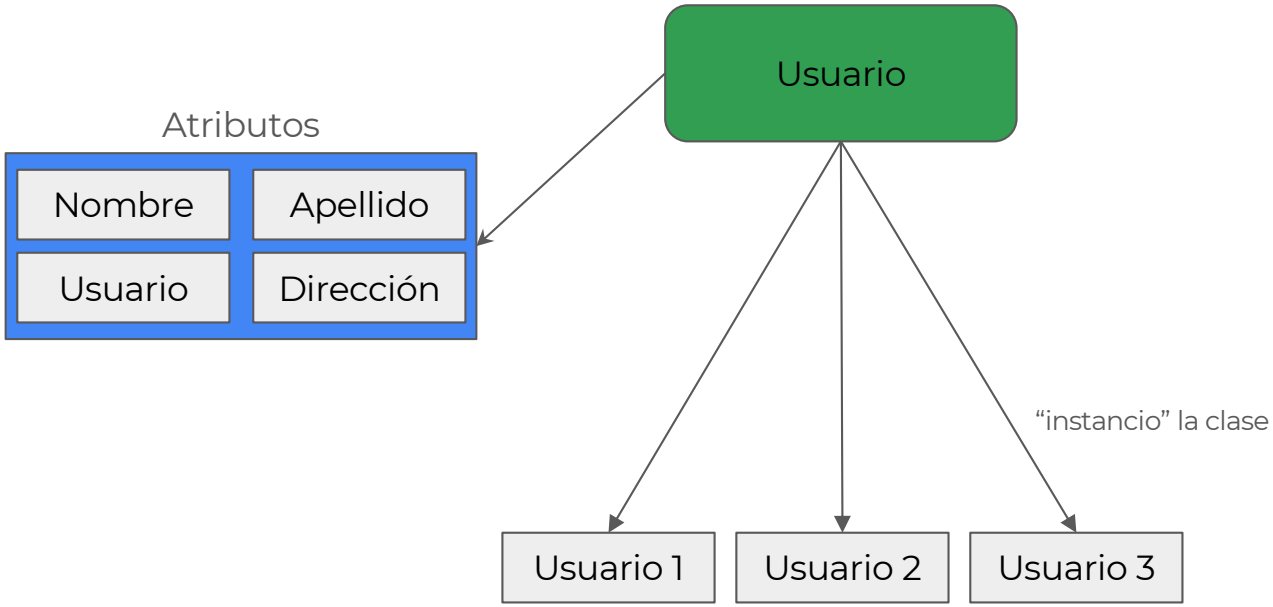
Es una instancia creada a partir de una clase. Cada objeto tiene sus propios valores en los atributos.



Conceptos Básicos de POO

Atributo

Son las características o datos que un objeto puede tener (variables dentro de la clase).

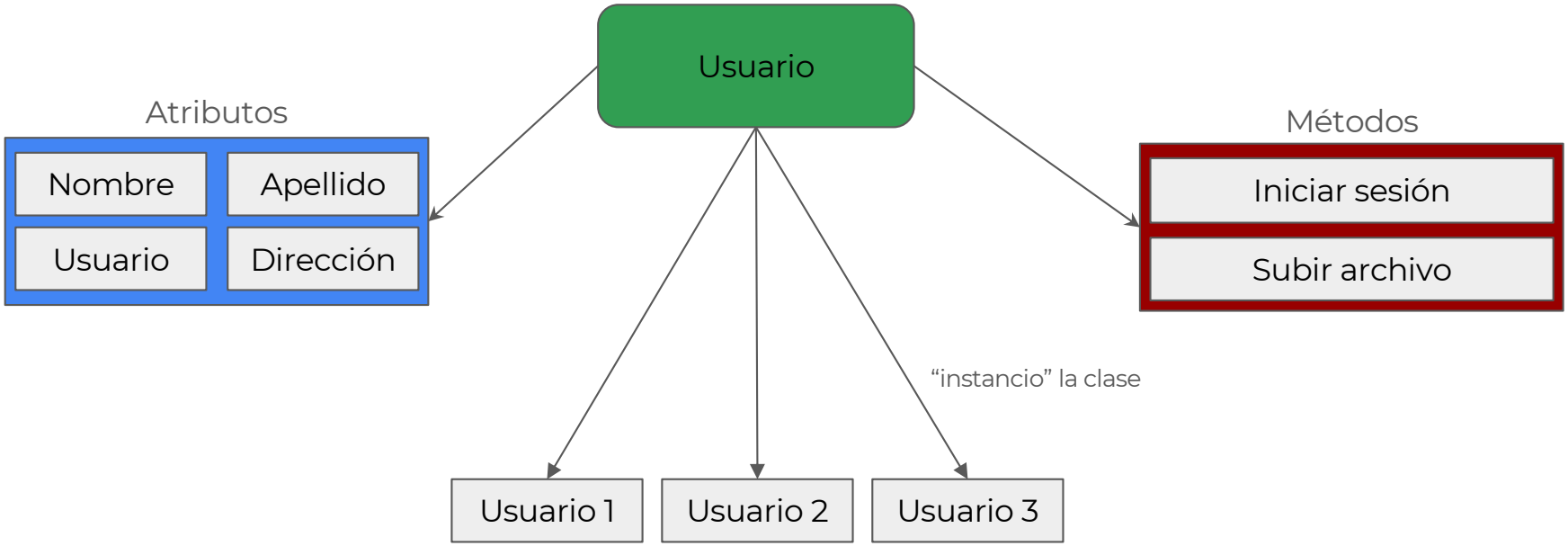


UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Conceptos Básicos de POO

Método

Son las funciones que definen el comportamiento del objeto (acciones que el objeto puede realizar).



Construcción de una Clase en Python

```
[in]  class NombreDeLaClase:
          def __init__(self): # Atributos de la clase
              pass
          def metodo(self): # Un método de la clase
              pass
```

Método __init__:
Es un **método especial** conocido como **constructor**. Se llama automáticamente cada vez que se crea una instancia de la clase (objeto). Su función principal es inicializar los atributos del objeto.

¿Cómo funciona?

- El primer parámetro del método __init__ es **SIEMPRE** self, que representa la instancia del objeto.
- Los otros parámetros permiten inicializar los atributos de la clase cuando el objeto es creado.

Construcción de una Clase en Python

```
[in]  class NombreDeLaClase:
          def __init__(self): # Atributos de la clase
              pass
          def metodo(self): # Un método de la clase
              pass
```

```
[in]  class Perro:
          def __init__(self, nombre, edad):
              self.nombre = nombre  # Atributo
              self.edad = edad      # Atributo

          def ladrar(self):
              print(f"{self.nombre} está ladrando.")
```

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Construcción de una Clase en Python

```
[in]  class Perro:
          def __init__(self, nombre, edad):
              self.nombre = nombre  # Atributo
              self.edad = edad      # Atributo

          def ladrar(self):
              print(f"{self.nombre} está ladrando.")

      mi_perro = Perro('Max', 6)
      mi_perro.ladrar()
```

```
[out] Max está ladrando.
```

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Otros *dunder methods*

```
[in]  class Perro:
          def __init__(self, nombre, edad):
              self.nombre = nombre  # Atributo
              self.edad = edad      # Atributo

          def __str__(self):
              return f"Nombre: {self.nombre}. Edad: {self.edad}"

      mi_perro = Perro('Max', 6)
      print(mi_perro)
```

```
[out] Nombre: Max. Edad: 6
```

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES